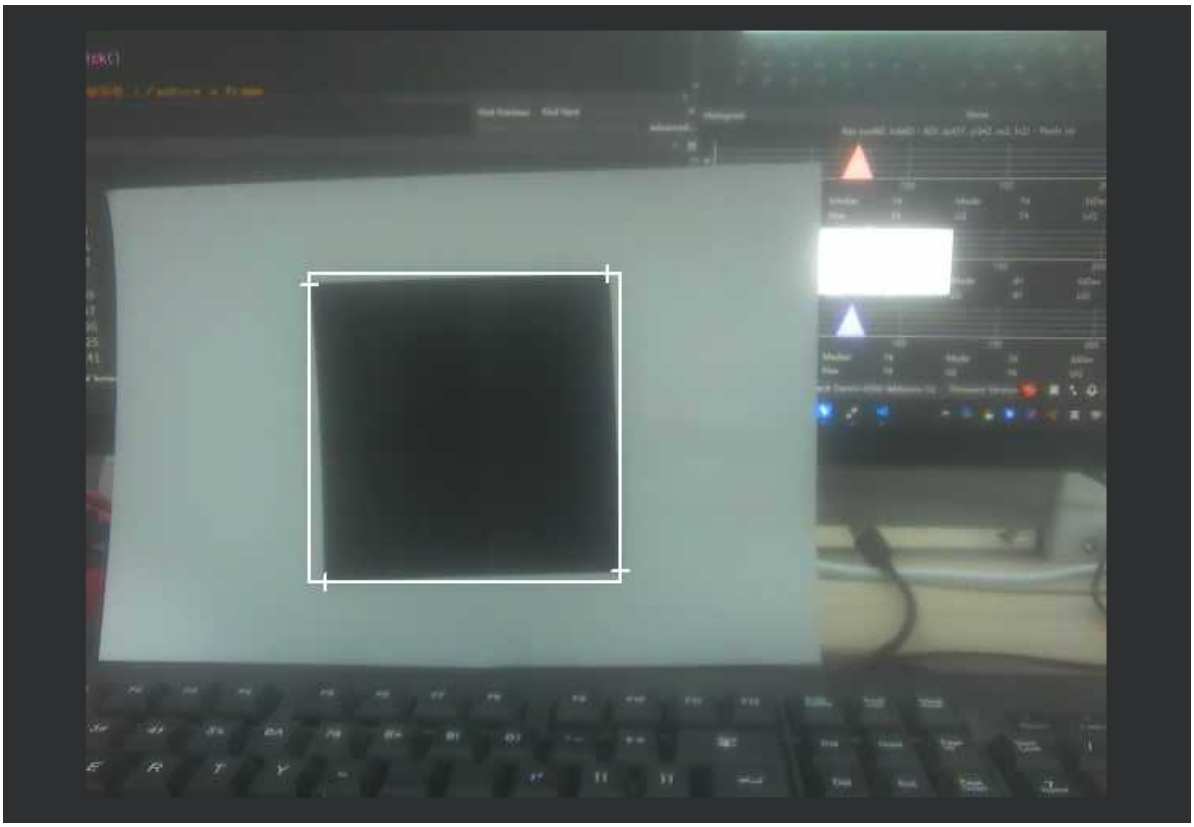
图像矩形检测

图像矩形检测
- 例程效果
- 原理讲解
  - 什么是图像矩形检测
- 代码概述
  - 导入模块
    - 设置图像尺寸
    - 初始化摄像头（RGB888 格式）
    - 初始化显示模块
    - 初始化媒体管理器并启动摄像头
    - 设置摄像头增益（亮度/对比度调节）
    - 设置帧率计时器
    - 设置矩形检测参数
    - 图像处理与矩形检测
    - 资源释放
  - 参数调整说明

例程效果

运行本节例程代码 [源码汇总/17.cv_lite/6.rgb888_find_rectangle_with_corners.py]

本节我们将结合 `cv_lite` 扩展模块在嵌入式设备上实现 RGB888 格式的图像矩形检测，并绘制矩形框和角点。



原理讲解

什么是图像矩形检测

图像矩形检测是一种计算机视觉技术，用于在图像中识别和定位矩形形状的对象。矩形检测通常基于边缘检测和轮廓分析，通过一系列图像处理步骤（如模糊、边缘提取、轮廓拟合等）来找到符合矩形特征的区域。本代码中使用 `cv_lite` 模块实现了矩形检测，并返回矩形的位置、大小以及四个角点的坐标。

- **操作原理**：矩形检测通常包括以下步骤：
  1. **预处理**：对图像进行高斯模糊以减少噪声干扰。
  2. **边缘检测**：使用 Canny 边缘检测算法提取图像中的边缘，基于两个阈值（低阈值和高阈值）来确定强边缘和弱边缘。
  3. **轮廓提取**：从边缘图像中提取轮廓，并对轮廓进行多边形拟合，筛选出接近矩形的形状。
  4. **矩形筛选**：根据面积比例、角度余弦等条件进一步筛选出符合矩形特征的轮廓，并计算矩形的边界框和角点坐标。
  5. **结果返回**：返回矩形的位置（x, y）、大小（宽度、高度）以及四个角点的坐标。
- **效果**：矩形检测能够准确识别图像中的矩形对象，并在图像上绘制矩形框和角点，适用于需要定位规则形状的场景。检测结果受参数（如 Canny 阈值、面积比例、角度余弦等）的影响，需根据实际图像特性调整。
- **应用场景**：矩形检测广泛应用于目标检测（如识别卡片、标志牌）、机器人视觉（如定位物体）、增强现实（AR）以及文档处理（如检测表格或页面边界）等领域。

在矩形检测中，参数的选择对检测精度和性能影响显著。例如，Canny 阈值决定了边缘检测的灵敏度，面积比例过滤了过小的矩形，角度余弦值控制了矩形的规则性。

代码概述

导入模块

```
import time, os, sys, gc
from machine import Pin
from media.sensor import *      # 摄像头接口 / Camera interface
from media.display import *     # 显示接口 / Display interface
from media.media import *       # 媒体资源管理器 / Media manager
import _thread
import cv_lite                  # cv_lite扩展模块 / cv_lite extension (C bindings)
import ulab.numpy as np         # MicroPython NumPy类库
```

设置图像尺寸

```
image_shape = [480, 640]  # 高 x 宽 / Height x Width
```

定义图像的分辨率为 480x640（高度 x 宽度），后续会根据此尺寸初始化摄像头和显示模块。

初始化摄像头（RGB888 格式）

```
sensor = Sensor(id=2, width=1280, height=960, fps=90)
sensor.reset()
sensor.set_framesize(width=image_shape[1], height=image_shape[0])
sensor.set_pixformat(Sensor.RGB888)  # RGB888格式 / rgb888 format
```

- 初始化摄像头，设置分辨率为 1280x960，帧率为 90 FPS。
- 将输出帧大小调整为 640x480，并设置为 RGB888 格式（三通道彩色图像，适合处理彩色图像的矩形检测）。

初始化显示模块

```
Display.init(Display.ST7701, width=image_shape[1], height=image_shape[0],
to_ide=True, quality=50)
```

初始化显示模块，使用 ST7701 驱动芯片，分辨率与图像尺寸一致。`to_ide=True` 表示图像会同时传输到 IDE 进行虚拟显示，`quality=50` 设置图像传输质量。

初始化媒体管理器并启动摄像头

```
MediaManager.init()
sensor.run()
```

初始化媒体资源管理器，并启动摄像头开始捕获图像流。

设置摄像头增益（亮度/对比度调节）

```
gain = k_sensor_gain()
gain.gain[0] = 20
sensor.again(gain)
```

设置摄像头的增益值（`gain[0] = 20`），用于调节图像的亮度和对比度，以优化图像质量，便于边缘检测。

设置帧率计时器

```
clock = time.clock()  # 启动帧率计时 / Start FPS timer
```

初始化帧率计时器，用于计算和显示每秒处理的帧数（FPS）。

设置矩形检测参数

```
canny_thresh1      = 50        # Canny 边缘检测低阈值 / Canny low threshold
canny_thresh2      = 150       # Canny 边缘检测高阈值 / Canny high threshold
approx_epsilon     = 0.04      # 多边形拟合精度比例（越小拟合越精确）/ Polygon
approximation accuracy
area_min_ratio     = 0.001     # 最小面积比例（相对于图像总面积）/ Min area ratio
max_angle_cos      = 0.3       # 最大角度余弦（越小越接近矩形）/ Max cosine of angle
between edges
gaussian_blur_size = 5         # 高斯模糊核尺寸（奇数）/ Gaussian blur kernel size
```

- `canny_thresh1` 和 `canny_thresh2`：Canny 边缘检测的低阈值和高阈值，用于控制边缘检测的灵敏度。低阈值用于连接弱边缘，高阈值用于确定强边缘。
- `approx_epsilon`：多边形拟合精度比例，值越小拟合越精确，但计算量增加。
- `area_min_ratio`：最小面积比例，相对于图像总面积，用于过滤过小的矩形。
- `max_angle_cos`：最大角度余弦值，用于判断轮廓是否接近矩形，值越小要求越接近直角。
- `gaussian_blur_size`：高斯模糊核尺寸，必须为奇数，用于预处理图像以减少噪声。

图像处理与矩形检测

```
while True:
    clock.tick()

    # 拍摄一帧图像 / Capture a frame
    img = sensor.snapshot()
    img_np = img.to_numpy_ref()

    # 调用底层矩形检测函数
    # 返回格式：[[x0, y0, w0, h0, c1.x, c1.y, c2.x, c2.y, c3.x, c3.y, c4,x, c4.y],
[x1, y1, w1, h1,c1.x, c1.y, c2.x, c2.y, c3.x, c3.y, c4,x, c4.y], ...]
    rects = cv_lite.rgb888_find_rectangles_with_corners(
        image_shape, img_np,
        canny_thresh1, canny_thresh2,
        approx_epsilon,
        area_min_ratio,
        max_angle_cos,
        gaussian_blur_size
    )
    # 遍历检测到的矩形并绘制矩形框和角点
    for i in range(len(rects)):
        r = rects[i]
        img.draw_rectangle(r[0], r[1], r[2], r[3], color=(255, 255, 255),
thickness=2)
        img.draw_cross(r[4], r[5], color=(255, 255, 255), size=5, thickness=2)
        img.draw_cross(r[6], r[7], color=(255, 255, 255), size=5, thickness=2)
        img.draw_cross(r[8], r[9], color=(255, 255, 255), size=5, thickness=2)
        img.draw_cross(r[10], r[11], color=(255, 255, 255), size=5, thickness=2)

    # 显示图像 / Show image
    Display.show_image(img)

    # 垃圾回收 & 输出帧率/ Garbage collect and print FPS
    gc.collect()
    print("fps:", clock.fps())
```

- **图像捕获**：通过 `sensor.snapshot()` 获取一帧图像，并通过 `to_numpy_ref()` 将其转换为 NumPy 数组引用。
- **矩形检测处理**：调用 `cv_lite.rgb888_find_rectangles_with_corners()` 函数执行矩形检测，返回检测到的矩形列表，每个矩形包含位置（x, y）、大小（宽度、高度）以及四个角点的坐标。
- **绘制矩形和角点**：遍历检测到的矩形列表，在图像上使用 `draw_rectangle()` 绘制矩形框，使用 `draw_cross()` 在每个角点位置绘制十字标记，颜色为白色。
- **图像显示**：将处理后的图像显示到屏幕或 IDE 虚拟窗口。
- **内存管理和帧率输出**：调用 `gc.collect()` 清理内存，并通过 `clock.fps()` 打印当前帧率。

资源释放

```
sensor.stop()
Display.deinit()
os.exitpoint(os.EXITPOINT_ENABLE_SLEEP)
time.sleep_ms(100)
MediaManager.deinit()
```

参数调整说明

- `canny_thresh1` 和 `canny_thresh2`：Canny 边缘检测阈值，低阈值（建议 30-70）用于连接弱边缘，高阈值（建议 100-200）用于确定强边缘。值过低可能导致过多噪声边缘，值过高可能错过真实边缘，需根据图像对比度调整。
- `approx_epsilon`：多边形拟合精度比例，值越小（如 0.01-0.05）拟合越精确，但计算量增加，建议从 0.02 开始测试。
- `area_min_ratio`：最小面积比例，值越大（如 0.01）过滤更多小矩形，值越小（如 0.001）保留更多小矩形，建议根据目标大小调整。
- `max_angle_cos`：最大角度余弦，值越小（如 0.1-0.3）要求轮廓更接近矩形（直角），值越大允许更多变形，建议从 0.2 开始测试。
- `gaussian_blur_size`：高斯模糊核尺寸，必须为奇数，值越大（如 5-9）去噪效果越强，但可能丢失细节，建议从 3 或 5 开始测试。